

## 第3章 数组和字符串

### 学习目标

- ☑ 掌握一维数组的声明和使用方法
- ☑ 掌握二维数组的声明和使用方法
- ☑ 掌握字符串的声明、赋值、比较和连接方法
- ☑ 熟悉字符的 ASCII 码和 `ctype.h` 中的字符函数
- ☑ 正确认识“++”、“+=”等能修改变量的运算符
- ☑ 掌握 `fgetc` 和 `getchar` 的使用方法
- ☑ 了解不同操作系统中换行符的表示方法
- ☑ 掌握 `fgets` 的使用方法并了解 `gets` 的“缓冲区溢出”漏洞
- ☑ 学会用常量表简化代码

第2章的程序很实用，也发挥出了计算机的计算优势，但没有发挥出计算机的存储优势——我们只用了屈指可数的变量。尽管有的程序也处理了大量的数据，但这些数据都只是“过客”，只参与了计算，并没有被保存下来。

本章介绍数组和字符串，二者都能保存大量的数据。字符串是一种数组（字符数组），但由于其应用的特殊性，适用一些特别的处理方式。

### 3.1 数 组

考虑这样一个问题：读入一些整数，逆序输出到一行中。已知整数不超过 100 个。如何编写这个程序呢？首先是循环读取输入。读入每个整数以后，应该做些什么呢？思来想去，在所有整数全部读完之前，似乎没有其他事可做。换句话说，只能把每个数都存下来。存放在哪里呢？答案是：数组。

程序 3-1 逆序输出

```
#include<stdio.h>
#define maxn 105
int a[maxn];
int main()
{
    int x, n = 0;
    while(scanf("%d", &x) == 1)
        a[n++] = x;
    for(int i = n-1; i >= 0; i--)
```

```

    printf("%d ", a[i]);
    printf("%d\n", a[0]);
    return 0;
}

```

语句“`int a[maxn]`”声明了一个包含 `maxn` 个整型变量的数组，它们是：`a[0]`, `a[1]`, `a[2]`, ..., `a[maxn-1]`。注意，没有 `a[maxn]`。

**提示 3-1：**语句“`int a[maxn]`”声明了一个包含 `maxn` 个整型变量的数组，即 `a[0]`, `a[1]`, ..., `a[maxn-1]`，但不包含 `a[maxn]`。`maxn` 必须是常数，不能是变量。

为什么这里声明 `maxn` 为 105 而不是 100 呢？因为这样更保险。

**提示 3-2：**在算法竞赛中，常常难以精确计算出需要的数组大小，数组一般会声明得稍大一些。在空间够用的前提下，浪费一点不会有太大影响。

接下来是语句“`a[n++] = x`”，它做了两件事：首先赋值 `a[n]=x`，然后执行 `n=n+1`。如果觉得难以理解，可以将其改写成“`{ a[n] = x; n=n+1; }`”。注意这里的花括号是不能省略的，因为在默认情况下，`for` 语句的循环体只有一条语句。只有使用花括号时，花括号里的语句才会整体作为循环体。一般地，当表达式里出现 `n++` 时，表达式会使用加 1 前的 `n` 计算表达式，当表达式计算完毕之后再给 `n` 加 1。

和 `n++` 相对应的，还有一个 `++n`，表示先给 `n` 增加 1，然后使用新的 `n`。前缀和后缀“++”运算符是 C 语言的特色之一。事实上，后面将要介绍的 C++ 语言名字里的“++”就是该运算符。

**提示 3-3：**对于变量 `n`，`n++` 和 `++n` 都会给 `n` 加 1，但当它们用在一个表达式中时，行为有所差别：`n++` 会使用加 1 前的值计算表达式，而 `++n` 会使用加 1 后的值计算表达式。“++”运算符是 C 语言的特色之一。

循环结束后，数据被存储在 `a[0]`, `a[1]`, ..., `a[n-1]` 中，其中变量 `n` 是整数的个数（想一想，为什么）。

存好以后就可以输出了：依次输出 `a[n-1]`, `a[n-2]`, ..., `a[1]` 和 `a[0]`。这里有一个小问题：一般要求输出的行首行尾均无空格，相邻两个数据间用单个空格隔开。这样，一共要输出 `n` 个整数，但只有 `n-1` 个空格，所以只好分两条语句输出。

在上述程序中，数组 `a` 被声明在 `main` 函数的外面。请试着把 `maxn` 定义中的 100 改成 1000000，比较一下把数组 `a` 放在 `main` 函数内外的运行结果是否相同。如果相同，试着把 1000000 改得再大一些。当实验完成之后，读者应该就能明白为什么要把 `a` 的定义放在 `main` 函数的外面了。简单地说，只有在放外面时，数组 `a` 才可以开得很大；放在 `main` 函数内时，数组稍大就会异常退出。其道理将在后面讨论，现在只需要记住规则即可。

**提示 3-4：**比较大的数组应尽量声明在 `main` 函数外，否则程序可能无法运行。

C 语言的数组并不是“一等公民”，而是“受歧视”的。例如，数组不能够进行赋值操作：在程序 3-1 中，如果声明的是“`int a[maxn], b[maxn]`”，是不能赋值 `b = a` 的。如果要从

数组  $a$  复制  $k$  个元素到数组  $b$ ，可以这样做：`memcpy(b, a, sizeof(int) * k)`。当然，如果数组  $a$  和  $b$  都是浮点型的，复制时要写成“`memcpy(b, a, sizeof(double) * k)`”。另外需要注意的是，使用 `memcpy` 函数要包含头文件 `string.h`。如果需要把数组  $a$  全部复制到数组  $b$  中，可以写得简单一些：`memcpy(b, a, sizeof(a))`。

**开灯问题。**有  $n$  盏灯，编号为  $1 \sim n$ 。第 1 个人把所有灯打开，第 2 个人按下所有编号为 2 的倍数的开关（这些灯将被关掉），第 3 个人按下所有编号为 3 的倍数的开关（其中关掉的灯将被打开，开着的灯将被关闭），依此类推。一共有  $k$  个人，问最后有哪些灯开着？输入  $n$  和  $k$ ，输出开着的灯的编号。 $k \leq n \leq 1000$ 。

样例输入：

7 3

样例输出：

1 5 6 7

### 【分析】

用  $a[1], a[2], \dots, a[n]$  表示编号为  $1, 2, 3, \dots, n$  的灯是否开着。模拟这些操作即可。

程序 3-2 开灯问题

```
#include<stdio.h>
#include<string.h>
#define maxn 1010
int a[maxn];
int main()
{
    int n, k, first = 1;
    memset(a, 0, sizeof(a));
    scanf("%d%d", &n, &k);
    for(int i = 1; i <= k; i++)
        for(int j = 1; j <= n; j++)
            if(j % i == 0) a[j] = !a[j];
    for(int i = 1; i <= n; i++)
        if(a[i]) { if(first) first = 0; else printf(" "); printf("%d", i); }
    printf("\n");
    return 0;
}
```

“`memset(a, 0, sizeof(a))`”的作用是把数组  $a$  清零，它也在 `string.h` 中定义。虽然也能用 `for` 循环完成相同的任务，但是用 `memset` 又方便又快捷。另一个技巧在输出：为了避免输出多余空格，设置了一个标志变量 `first`，可以表示当前要输出的变量是否为第一个。第一个变量前不应有空格，但其他变量都有。

**蛇形填数。**在  $n \times n$  方阵里填入  $1, 2, \dots, n \times n$ ，要求填成蛇形。例如， $n=4$  时方阵为：

```

10 11 12 1
9 16 13 2
8 15 14 3
7 6 5 4

```

上面的方阵中, 多余的空格只是为了便于观察规律, 不必严格输出。 $n \leq 8$ 。

### 【分析】

类比数学中的矩阵, 可以用一个二维数组来储存题目中的方阵。只需声明一个“`int a[maxn][maxn]`”, 就可以获得一个大小为  $\text{maxn} \times \text{maxn}$  的方阵。在声明时, 二维的大小不必相同, 因此也可以声明 `int a[30][50]` 这样的数组, 第一维下标范围是  $0, 1, 2, \dots, 29$ , 第二维下标范围是  $0, 1, 2, \dots, 49$ 。

**提示 3-5:** 可以用“`int a[maxn][maxm]`”生成一个整型的二维数组, 其中  $\text{maxn}$  和  $\text{maxm}$  不必相等。这个数组共有  $\text{maxn} \times \text{maxm}$  个元素, 分别为 `a[0][0]`, `a[0][1]`,  $\dots$ , `a[0][maxm-1]`, `a[1][0]`, `a[1][1]`,  $\dots$ , `a[1][maxm-1]`,  $\dots$ , `a[maxn-1][0]`, `a[maxn-1][1]`,  $\dots$ , `a[maxn-1][maxm-1]`。

从 1 开始依次填写。设“笔”的坐标为  $(x, y)$ , 则一开始  $x=0$ ,  $y=n-1$ , 即第 0 行, 第  $n-1$  列 (行列的范围是  $0 \sim n-1$ , 没有第  $n$  列)。“笔”的移动轨迹是: 下, 下, 下, 左, 左, 左, 上, 上, 上, 右, 右, 下, 下, 左, 上。总之, 先是下, 到不能填为止, 然后是左, 接着是上, 最后是右。“不能填”是指再走就出界 (例如  $4 \rightarrow 5$ ), 或者再走就要走到以前填过的格子 (例如  $12 \rightarrow 13$ )。如果把所有格子初始化为 0, 就能很方便地加以判断。

### 程序 3-3 蛇形填数

```

#include<stdio.h>
#include<string.h>
#define maxn 20
int a[maxn][maxn];
int main()
{
    int n, x, y, tot = 0;
    scanf("%d", &n);
    memset(a, 0, sizeof(a));
    tot = a[x=0][y=n-1] = 1;
    while(tot < n*n)
    {
        while(x+1<n && !a[x+1][y]) a[++x][y] = ++tot;
        while(y-1>=0 && !a[x][y-1]) a[x][--y] = ++tot;
        while(x-1>=0 && !a[x-1][y]) a[--x][y] = ++tot;
        while(y+1<n && !a[x][y+1]) a[x][++y] = ++tot;
    }
    for(x = 0; x < n; x++)
    {

```

```

    for(y = 0; y < n; y++) printf("%3d", a[x][y]);
    printf("\n");
}
return 0;
}

```

这段程序充分利用了 C 语言简洁的优势。首先,赋值  $x=0$  和  $y=n-1$  后马上要把它们作为数组  $a$  的下标,因此可以合并完成; $tot$  和  $a[0][n-1]$  都要赋值 1,也可以合并完成。这样,就用一条语句完成了多件事情,而且并没有牺牲程序的可读性——这段代码的含义显而易见。

**提示 3-6:** 可以利用 C 语言简洁的语法,但前提是保持代码的可读性。

那 4 条 `while` 语句有些难懂,不过十分相似,因此只需介绍其中的第一条:不断向下走,并且填数。我们的原则是:先判断,再移动,而不是走一步以后发现越界了再退回来。这样,则需要进行“预判”,即是否越界,以及如果继续往下走会不会到达一个已经填过的格子。越界只需判断  $x+1 < n$ ,因为  $y$  的值并没有修改;下一个格子是  $(x+1, y)$ ,因此只需“ $a[x+1][y] == 0$ ”,简写成“ $!a[x+1][y]$ ”(其中“ $!$ ”是“逻辑非”运算符)。

**提示 3-7:** 在很多情况下,最好是在做一件事之前检查是不是可以做,而不要做完再后悔。因为“悔棋”往往比较麻烦。

细心的读者也许会发现这里的一个“潜在 bug”:如果越界,  $x+1$  会等于  $n$ ,  $a[x+1][y]$  将访问非法内存!幸运的是,这样的担心是不必要的。“ $\&\&$ ”是短路运算符(还记得我们在那里提到过吗?)。如果  $x+1 < n$  为假,将不会计算“ $!a[x+1][y]$ ”,也就不会越界了。

至于为什么是  $++tot$  而不是  $tot++$ ,留给读者思考。

## 3.2 字符数组

文本处理在计算机应用中占有重要地位。本书到现在为止还没有正式讨论过字符串(尽管曾经使用过),因为在 C 语言中,字符串其实就是字符数组——可以像处理普通数组一样处理字符串,只需要注意输入输出和字符串函数的使用。

**竖式问题。**找出所有形如  $abc*de$  (三位数乘以两位数)的算式,使得在完整的竖式中,所有数字都属于一个特定的数字集合。输入数字集合(相邻数字之间没有空格),输出所有竖式。每个竖式前应有编号,之后应有一个空行。最后输出解的总数。具体格式见样例输出(为了便于观察,竖式中的空格改用小数点显示,但所写程序中应该输出空格,而非小数点)。

样例输入:

2357

样例输出：

```
<1>
..775
X..33
-----
.2325
2325.
-----
25575
```

The number of solutions = 1

### 【分析】

本题的思路应该是很清晰的：尝试所有的  $abc$  和  $de$ ，判断是否满足条件。我们可以写出整个程序的伪代码：

```
char s[20];
int count = 0;
scanf("%s", s);
for(int abc = 111; abc <= 999; abc++)
    for(int de = 11; de <= 99; de++)
        if("abc*de"是个合法的竖式)
        {
            printf("<%d>\n", count);
            打印 abc*de 的竖式和其后的空行
            count++;
        }
printf("The number of solutions = %d\n", count);
```

第一个新内容是 `char s[20]` 定义。`char` 是“字符型”的意思，而字符是一种特殊的整数。为什么字符会是特殊的整数？请参见图 3-1 所示的 ASCII 编码表。

|     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 0   | NUL | 1   | SOH | 2   | STX | 3   | ETX | 4   | EOT | 5   | ENQ | 6   | ACK | 7   | BEL |
| 8   | BS  | 9   | HT  | 10  | NL  | 11  | VT  | 12  | NP  | 13  | CR  | 14  | SO  | 15  | SI  |
| 16  | DLE | 17  | DC1 | 18  | DC2 | 19  | DC3 | 20  | DC4 | 21  | NAK | 22  | SYN | 23  | ETB |
| 24  | CAN | 25  | EM  | 26  | SUB | 27  | ESC | 28  | FS  | 29  | GS  | 30  | RS  | 31  | US  |
| 32  | SP  | 33  | !   | 34  | "   | 35  | #   | 36  | \$  | 37  | %   | 38  | &   | 39  | '   |
| 40  | (   | 41  | )   | 42  | *   | 43  | +   | 44  | ,   | 45  | -   | 46  | .   | 47  | /   |
| 48  | 0   | 49  | 1   | 50  | 2   | 51  | 3   | 52  | 4   | 53  | 5   | 54  | 6   | 55  | 7   |
| 56  | 8   | 57  | 9   | 58  | :   | 59  | ;   | 60  | <   | 61  | =   | 62  | >   | 63  | ?   |
| 64  | @   | 65  | A   | 66  | B   | 67  | C   | 68  | D   | 69  | E   | 70  | F   | 71  | G   |
| 72  | H   | 73  | I   | 74  | J   | 75  | K   | 76  | L   | 77  | M   | 78  | N   | 79  | O   |
| 80  | P   | 81  | Q   | 82  | R   | 83  | S   | 84  | T   | 85  | U   | 86  | V   | 87  | W   |
| 88  | X   | 89  | Y   | 90  | Z   | 91  | [   | 92  | /   | 93  | ]   | 94  | -   | 95  | _   |
| 96  | `   | 97  | a   | 98  | b   | 99  | {   | 100 | d   | 101 | e   | 102 | f   | 103 | g   |
| 104 | h   | 105 | i   | 106 | j   | 107 | k   | 108 | l   | 109 | m   | 110 | n   | 111 | o   |
| 112 | p   | 113 | q   | 114 | r   | 115 | e   | 116 | t   | 117 | u   | 118 | v   | 119 | w   |
| 120 | x   | 121 | y   | 122 | z   | 123 | {   | 124 | --  | 125 | }   | 126 | ~   | 127 | DEL |

图 3-1 ASCII 编码表

从图 3-1 中可见，每一个字符都有一个整数编码，称为 ASCII 码。为了方便书写，C 语

言允许用直接的方法表示字符，例如，“a”代表的就是a的ASCII码。不过，有一些字符直接表示出来并不方便，例如，回车符是“\n”，而空字符是“\0”，它也是C语言中字符串的结束标志。其他例子包括“\\”（注意必须有两个反斜线）、“\'”（这个是单引号），甚至还有的字符有两种写法：“\”和“”都表示双引号。像这种以反斜线开头的字符称为转义序列（Escape Sequence）。如果认真完成了第1章中的实验，相信对这些字符不会陌生。

**提示 3-8：**C语言中的字符型用关键字char表示，它实际存储的是字符的ASCII码。字符常量可以用单引号法表示。在语法上可以把字符当作int型使用。

另一个新内容是“scanf("%s", s)”。和“scanf("%d", &n)”类似，它会读入一个不含空格、TAB和回车符的字符串，存入字符数组s。注意，不是“scanf("%s", &s)”，s前面没有“&”符号。

**提示 3-9：**在“scanf("%s", s)”中，不要在s前面加上“&”符号。如果是字符串数组char s[maxn][maxl]，可以用“scanf("%s", s[i])”读取第i个字符串。注意，“scanf("%s", s)”遇到空白字符会停下来。

接下来有两个问题：判断和输出。根据我们的一贯作风，先考虑输出，因为它比较简单。每个竖式需要打印7行，但不一定要用7条printf语句，1条足矣。首先计算第一行乘积 $x=abc*e$ ，然后是第二行 $y=abc*d$ ，最后是总乘积 $z=abc*de$ ，然后一次性打印出来：

```
printf("%5d\nX%4d\n-----\n%5d\n%4d\n-----\n%5d\n\n", abc, de, x, y, z);
```

注意这里的%5d，它表示按照5位数打印，不足5位在前面补空格（还记得%03d吗？）。完整程序如下：

程序 3-4 竖式问题

```
#include<stdio.h>
#include<string.h>
int main()
{
    int count = 0;
    char s[20], buf[99];
    scanf("%s", s);
    for(int abc = 111; abc <= 999; abc++)
        for(int de = 11; de <= 99; de++)
        {
            int x = abc*(de%10), y = abc*(de/10), z = abc*de;
            sprintf(buf, "%d%d%d%d", abc, de, x, y, z);
            int ok = 1;
            for(int i = 0; i < strlen(buf); i++)
                if(strchr(s, buf[i]) == NULL) ok = 0;
            if(ok)
```

```

    {
        printf("<%d>\n", ++count);
        printf("%5d\nX%4d\n-----\n%5d\n%4d\n-----\n%5d\n\n", abc, de, x, y, z);
    }
}

printf("The number of solutions = %d\n", count);
return 0;
}

```

还有两个函数是以前没有遇到的: `sprintf` 和 `strchr`。 `strchr` 的作用是在一个字符串中查找单个字符, 而这个 `sprintf` 似曾相识: 之前用过 `printf` 和 `fprintf`。没错! 这 3 个函数是“亲兄弟”, `printf` 输出到屏幕, `fprintf` 输出到文件, 而 `sprintf` 输出到字符串。多数情况下, 屏幕总是可以输出的, 文件一般也能写 (除非磁盘满或者硬件损坏), 但字符串就不一定了: 应该保证写入的字符串有足够的空间。

**提示 3-10:** 可以用 `sprintf` 把信息输出到字符串, 用法和 `printf`、`fprintf` 类似。但应当保证字符串足够大, 可以容纳输出信息。

多大才算足够大呢? 答案是字符个数加 1, 因为 C 语言的字符串是以空字符 “\0” 结尾的。后面还会提到这个问题, 但是基本原则仍然是以前说过的: 如果算不清楚就把数组空间设置得大一点, 空间够用的情况下浪费一点没关系。例如, 此处声明的缓冲字符串 `buf` 的长度为 99 (可以保存长度为 98 的字符串), 保存 `abc, de, x, y, z` 的所有数字绰绰有余。

函数 `strlen(s)` 的作用是获取字符串 `s` 的实际长度。什么叫实际长度呢? 字符数组 `s` 的大小是 20, 但并不是所有空间都用上了。如果输入是 “2357”, 那么实际上 `s` 只保存了 5 个字符 (不要忘记了还有一个结束标记 “\0”), 后面 15 个字符是不确定的 (还记得吗? 变量在赋值之前是不确定的)。`strlen(s)` 返回的就是结束标记之前的字符个数。因此这个字符串中的各个字符依次是 `s[0], s[1], ..., s[strlen(s)-1]`, 而 `s[strlen(s)]` 正是结束标记 “\0”。

**提示 3-11:** C 语言中的字符串是以 “\0” 结尾的字符数组, 可以用 `strlen(s)` 返回字符串 `s` 中结束标记之前的字符个数。字符串中的各个字符是 `s[0], s[1], ..., s[strlen(s)-1]`。

**提示 3-12:** 由于字符串的本质是数组, 它也不是 “一等公民”, 只能用 `strcpy(a, b)`, `strcmp(a, b)`, `strcat(a, b)` 来执行 “赋值”、“比较” 和 “连接” 操作, 而不能用 “=”、“==”、“<=”、“+” 等运算符。上述函数都在 `string.h` 中声明。

此处再次看到了 `++count` 这样的用法, 有必要对它进行进一步说明。猜猜看: `count=0` 时, “`printf("%d %d %d\n", count++, count++, count++)`” 会输出什么 (然后做个实验)。怎么样, 是不是和你想的不同呢?

另一个例子是 “`count = count++`”。这里对 `count++` 的解释是: `count++` 在表达式中的值是加 1 之前的值 (即原来的值), 但计算 `count++` 之后 `count` 会增加 1。问题出现了: 这个 “稍后再加 1” 到底是何时进行的呢? 如果是计算完赋值的右边 (即 `count++`) 之后



就立刻执行，最后 `count` 的值不会变（别忘了最后执行的是赋值）；但如果是整个赋值完成之后才加 1，最后 `count` 的值会比原来多 1。如果在理解刚才这段话时感到吃力，最好的方法是避开它。

**提示 3-13：**滥用“++”、“--”、“+=”等可以修改变量值的运算符很容易带来隐蔽的错误。建议每条语句最多只用一次这种运算符，并且所修改的变量在整条语句中只出现一次。

事实上，就算充分理解了这条规则，在实际编程时也可能临时忘记。好在可以利用编译器减少这种错误。用 `-Wall` 命令编译刚才的两个例子，编译器都会给出警告：对 `count` 的运算可能是没有定义的。

### 3.3 竞赛题目选讲

#### 例题 3-1 TeX 中的引号 (Tex Quotes, UVa 272)<sup>①</sup>

在 TeX 中，左双引号是“````”，右双引号是“`''`”。输入一篇包含双引号的文章，你的任务是把它转换成 TeX 的格式。

样例输入：

```
"To be or not to be," quoth the Bard, "that
is the question".
```

样例输出：

```
``To be or not to be, '' quoth the Bard, ``that
is the question''.
```

#### 【分析】

本题的关键是，如何判断一个双引号是左双引号还是右双引号。方法很简单：使用一个标志变量即可。可是在在此之前，需要解决另外一个问题：输入字符串。

之前学习了使用“`scanf("%s")`”输入字符串，但却不能在本题中使用它，因为它碰到空格或者 TAB 就会停下来。虽然下次调用时会输入下一个字符串，可是不知道两次输入的字符串中间有多少个空格、TAB 甚至换行符。可以用下述两种方法解决这个问题：

第一种方法是使用“`fgetc(fin)`”，它读取一个打开的文件 `fin`，读取一个字符，然后返回一个 `int` 值。为什么返回的是 `int` 而不是 `char` 呢？因为如果文件结束，`fgetc` 将返回一个特殊标记 EOF，它并不是一个 `char`。如果把 `fgetc(fin)` 的返回值强制转换为 `char`，将无法把特殊的 EOF 和普通字符区分开。如果要从标准输入读取一个字符，可以用 `getchar`，它等价于 `fgetc(stdin)`。

**提示 3-14：**使用 `fgetc(fin)` 可以从打开的文件 `fin` 中读取一个字符。一般情况下应当在检查它

<sup>①</sup> 本章最后会介绍 UVaXXX 的含义。

不是 EOF 后再将其转换成 char 值。从标准输入读取一个字符可以用 `getchar`，它等价于 `fgetc(stdin)`。

`fgetc` 和 `getchar` 将读取“下一个字符”，因此需要知道在各种情况下，“下一个字符”是哪个。如果用“`scanf("%d", &n)`”读取整数 `n`，则要是输入 123 后多加了一个空格，用 `getchar` 读取的将是这个空格；如果在“123”之后紧跟着换行，则读取到的将是回车符“`\n`”。

这里有个潜在的陷阱：不同操作系统的回车换行符是不一致的。Windows 是“`\r`”和“`\n`”两个字符，Linux 是“`\n`”，而 MacOS 是“`\r`”。如果在 Windows 下读取 Windows 文件，`fgetc` 和 `getchar` 会把“`\r`”“吃掉”，只剩下“`\n`”；但如果要在 Linux 下读取同样一个文件，它们会忠实地先读取“`\r`”，然后才是“`\n`”。如果编程时不注意，所写程序可能会在某个操作系统上是完美的，但在另一个操作系统上就错得一塌糊涂。当然，比赛的组织方应该避免在 Linux 下使用 Windows 格式的文件，但正如前面所强调过的：选手也应该把自己的程序写得更鲁棒，即容错性更好。

**提示 3-15：**在使用 `fgetc` 和 `getchar` 时，应该避免写出和操作系统相关的程序。

第二种方法是使用“`fgets(buf, maxn, fin)`”读取完整的一行，其中 `buf` 的声明为 `char buf[maxn]`。这个函数读取不超过 `maxn-1` 个字符，然后在末尾添上结束符“`\0`”，因此不会出现越界的情况。之所以说可以用这个函数读取完整的一行，是因为一旦读到回车符“`\n`”，读取工作将会停止，而这个“`\n`”也会是 `buf` 字符串中最后一个有效字符（再往后就是字符串结束符“`\0`”了）。只有在一种情况下，`buf` 不会以“`\n`”结尾：读到文件结束符，并且文件的最后一个不是以“`\n`”结尾。尽管比赛的组织方应避免这样的情况（和输出文件一样，保证输入文件的每行均以回车符结尾），但正如刚才所说，选手应该把自己的程序写得更鲁棒。

**提示 3-16：**“`fgets(buf, maxn, fin)`”将读取完整的一行放在字符数组 `buf` 中。应当保证 `buf` 足够存放下文件的一行内容。除了在文件结束前没有遇到“`\n`”这种特殊情况外，`buf` 总是以“`\n`”结尾。当一个字符都没有读到时，`fgets` 返回 `NULL`。

和 `fgetc` 一样，`fgets` 也有一个“标准输入版”`gets`。遗憾的是，`gets` 和它的“兄弟”`fgets` 差别比较大：其用法是 `gets(s)`，没有指明读取的最大字符数。这里就出现了一个潜在的问题：`gets` 将不停地往 `s` 中存储内容，而不管是否存储得下！难道 `gets` 函数不去管 `s` 的可用空间有多少吗？确实如此。

**提示 3-17：**C 语言并不禁止程序读写“非法内存”。例如，声明的是 `char s[100]`，完全可以赋值 `s[10000]='a'`（甚至 `-Wall` 也不会警告），但后果自负。

正是因为如此，`gets` 已经被废除了，但为了向后兼容，仍然可以使用它。从长远考虑，读者最好不要使用此函数。事实上，在 C11 标准里，`gets` 函数已被正式删除。

**提示 3-18：**C 语言中的 `gets(s)` 存在缓冲区溢出漏洞，不推荐使用。在 C11 标准里，该函数已被正式删除。

本题的特点是：可以边读边处理，而不需要把输入字符串完整地存下来，因此 `getchar` 是一个不错的选择。下面的代码里还有一个有趣的运算符“?:”，是 `if` 语句的“表达式版”。表达式“`a?b:c`”的含义是：当 `a` 为真时值为 `b`，否则为 `c`。另一个细节是直接用到了赋值语句“`c=getchar()`”的返回值，把它和 `EOF` 进行比较。这样的写法并不多见，但有时能让代码更简洁。

程序 3-5 TeX 中的引号

```
#include<stdio.h>

int main() {
    int c, q = 1;
    while((c = getchar()) != EOF) {
        if(c == '"') { printf("%s", q ? "\"" : "'"); q = !q; }
        else printf("%c", c);
    }
    return 0;
}
```

### 例题 3-2 WERTYU (WERTYU, UVa10082)

把手放在键盘上时，稍不注意就会往右错一位。这样，输入 `Q` 会变成输入 `W`，输入 `J`会变成输入 `K` 等。键盘如图 3-2 所示。

输入一个错位后敲出的字符串（所有字母均大写），输出打字员本来想打出的句子。输入保证合法，即一定是错位之后的字符串。例如输入中不会出现大写字母 `A`。

样例输入：

```
O S, GOMR YPFSU/
```

样例输出：

```
I AM FINE TODAY.
```

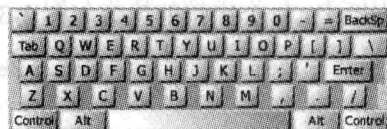


图 3-2 键盘

#### 【分析】

和例题 3-1 一样，每输入一个字符，都可以直接输出一个字符，因此 `getchar` 是输入的理想方法。问题在于：如何进行这样输入输出变换呢？一种方法是使用 `if` 语句或者 `switch` 语句，如“`if(c == 'W') putchar('Q')`”。但很明显，这样做太麻烦。一个较好的方法是使用常量数组，下面是完整程序：

程序 3-6 WERTYU

```
#include<stdio.h>

char s[] = "`1234567890-=QWERTYUIOP[]\\ASDFGHJKL;'ZXCVBNM,./";

int main() {
```

```
int i, c;
while((c = getchar()) != EOF) {
    for (i=1; s[i] && s[i]!=c; i++); //找错位之后的字符在常量表中的位置
    if (s[i]) putchar(s[i-1]); //如果找到，则输出它的前一个字符
    else putchar(c);
}
return 0;
}
```

还有其他使用常量数组的方法。例如，构造一个数组  $s$ ，使得对于任意字符  $c$ ， $s[c]$  的值为  $c$  “左边” 的字符。这个方法也是可行的，但是在程序里输入这样一个  $s$  数组有些麻烦，还是本题的策略更容易实现。常量数组并不需要指明大小，编译器可以完成计算。

**提示 3-19：**善用常量数组往往能简化代码。定义常量数组时无须指明大小，编译器会计算。

### 例题 3-3 回文词 (Palindromes, UVa401)

输入一个字符串，判断它是否为回文串以及镜像串。输入字符串保证不含数字 0。所谓回文串，就是反转以后和原串相同，如 *abba* 和 *madam*。所有镜像串，就是左右镜像之后和原串相同，如 *2S* 和 *3AIAE*。注意，并不是每个字符在镜像之后都能得到一个合法字符。在本题中，每个字符的镜像如图 3-3 所示（空白项表示该字符镜像后不能得到一个合法字符）。

| Character | Reverse | Character | Reverse | Character | Reverse |
|-----------|---------|-----------|---------|-----------|---------|
| A         | A       | M         | M       | Y         | Y       |
| B         |         | N         |         | Z         | S       |
| C         |         | O         | O       | 1         | 1       |
| D         |         | P         |         | 2         | S       |
| E         | 3       | Q         |         | 3         | E       |
| F         |         | R         |         | 4         |         |
| G         |         | S         | 2       | 5         | Z       |
| H         | H       | T         | T       | 6         |         |
| I         | I       | U         | U       | 7         |         |
| J         | L       | V         | V       | 8         | 8       |
| K         |         | W         | W       | 9         |         |
| L         | J       | X         | X       |           |         |

图 3-3 镜像字符

输入的每行包含一个字符串（保证只有上述字符。不含空白字符），判断它是否为回文串和镜像串（共 4 种组合）。每组数据之后输出一个空行。

样例输入：

```
NOTAPALINDROME
ISAPALINILAPASI
2A3MEAS
ATOYOTA
```

样例输出:

```
NOTAPALINDROME -- is not a palindrome.
```

```
ISAPALINILAPASI -- is a regular palindrome.
```

```
2A3MEAS -- is a mirrored string.
```

```
ATOYOTA -- is a mirrored palindrome.
```

### 【分析】

既然不包含空白字符,可以安全地使用 `scanf` 进行输入。回文串和镜像串的判断都不复杂,并且可以一起完成,详见下面的代码。使用常量数组,只用少量代码即可解决这个看上去有些复杂的题目<sup>①</sup>。

程序 3-7 回文词

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>
const char* rev = "A 3 HIL JM O 2TUVWXY51SE Z 8 ";
const char* msg[] = {"not a palindrome", "a regular palindrome", "a mirrored
string", "a mirrored palindrome"};

char r(char ch) {
    if(isalpha(ch)) return rev[ch - 'A'];
    return rev[ch - '0' + 25];
}

int main() {
    char s[30];
    while(scanf("%s", s) == 1) {
        int len = strlen(s);
        int p = 1, m = 1;
        for(int i = 0; i < (len+1)/2; i++) {
            if(s[i] != s[len-1-i]) p = 0; //不是回文串
            if(r(s[i]) != s[len-1-i]) m = 0; //不是镜像串
        }
        printf("%s -- is %s.\n\n", s, msg[m*2+p]);
    }
    return 0;
}
```

<sup>①</sup> 本题是《算法竞赛入门经典》第1版中的一道习题。在第2版写作之时,笔者在网上搜到了很多网友写的本题的题解,不少博主表示本题比较麻烦或者代码冗长,容易写错,故而将此题补充到第2版的例题当中。

本题使用了一个自定义函数 `char r(char ch)`, 参数 `ch` 是一个字符, 返回值是 `ch` 的镜像字符。这是因为该常量数组中前 26 项是各个大写字母的镜像, 而后 10 个是数字 1~9 的镜像 (数字 0 不会出现), 所以需要判断 `ch` 是字母还是数字。函数在第 4 章中会详细讨论, 如果现在理解有困难, 可以等看完第 4 章后回顾此题。

本题用 `isalpha` 来判断字符是否为字母, 类似的还有 `isdigit`、`isprint` 等, 在 `ctype.h` 中定义。由于 ASCII 码表中大写字母、小写字母和数字都是连续的, 如果 `ch` 是大写字母, 则 `ch-'A'` 就是它在字母表中的序号 (A 的序号是 0, B 的序号是 1, 依此类推); 类似地, 如果 `ch` 是数字, 则 `ch-'0'` 就是这个数字的数值本身 (例如 `'5'-'0'=5`)。

另一个有趣的常量数组是 `msg` (事实上, 这是一个字符串数组, 即二维字符数组), 请读者自行理解它的作用。

**提示 3-20:** 头文件 `ctype.h` 中定义的 `isalpha`、`isdigit`、`isprint` 等工具可以用来判断字符的属性, 而 `toupper`、`tolower` 等工具可以用来转换大小写。如果 `ch` 是大写字母, 则 `ch-'A'` 就是它在字母表中的序号 (A 的序号是 0, B 的序号是 1, 依此类推); 类似地, 如果 `ch` 是数字, 则 `ch-'0'` 就是这个数字的数值本身。

### 例题 3-4 猜数字游戏的提示 (Master-Mind Hints, UVa 340)

实现一个经典“猜数字”游戏。给定答案序列和用户猜的序列, 统计有多少数字位置正确 (A), 有多少数字在两个序列都出现过但位置不对 (B)。

输入包含多组数据。每组输入第一行为序列长度  $n$ , 第二行是答案序列, 接下来是若干猜测序列。猜测序列全 0 时该组数据结束。 $n=0$  时输入结束。

样例输入:

```
4
1 3 5 5
1 1 2 3
4 3 3 5
6 5 5 1
6 1 3 5
1 3 5 5
0 0 0 0
10
1 2 2 2 4 5 6 6 6 9
1 2 3 4 5 6 7 8 9 1
1 1 2 2 3 3 4 4 5 5
1 2 1 3 1 5 1 6 1 9
1 2 2 5 5 5 6 6 6 7
0 0 0 0 0 0 0 0 0 0
0
```

样例输出:

Game 1:

(1,1)

(2,0)

(1,2)

(1,2)

(4,0)

Game 2:

(2,4)

(3,2)

(5,0)

(7,0)

### 【分析】

直接统计可得 A, 为了求 B, 对于每个数字 (1~9), 统计二者出现的次数  $c_1$  和  $c_2$ , 则  $\min(c_1, c_2)$  就是该数字对 B 的贡献。最后要减去 A 的部分。代码如下:

```
#include<stdio.h>
```

```
#define maxn 1010
```

```
int main() {
```

```
    int n, a[maxn], b[maxn];
```

```
    int kase = 0;
```

```
    while(scanf("%d", &n) == 1 && n) { //n=0 时输入结束
```

```
        printf("Game %d:\n", ++kase);
```

```
        for(int i = 0; i < n; i++) scanf("%d", &a[i]);
```

```
        for(;;) {
```

```
            int A = 0, B = 0;
```

```
            for(int i = 0; i < n; i++) {
```

```
                scanf("%d", &b[i]);
```

```
                if(a[i] == b[i]) A++;
```

```
            }
```

```
            if(b[0] == 0) break; //正常的猜测序列不会有 0, 所以只判断第一个数是否为 0 即可
```

```
            for(int d = 1; d <= 9; d++) {
```

```
                int c1 = 0, c2 = 0; //统计数字 d 在答案序列和猜测序列中各出现多少次
```

```
                for(int i = 0; i < n; i++) {
```

```
                    if(a[i] == d) c1++;
```

```
                    if(b[i] == d) c2++;
```

```
                }
```

```
                if(c1 < c2) B += c1; else B += c2;
```

```
            }
```

```
            printf("    (%d,%d)\n", A, B-A);
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

**例题 3-5 生成元 (Digit Generator, ACM/ICPC Seoul 2005, UVa1583)**

如果  $x$  加上  $x$  的各个数字之和得到  $y$ , 就说  $x$  是  $y$  的生成元。给出  $n$  ( $1 \leq n \leq 100000$ ), 求最小生成元。无解输出 0。例如,  $n=216$ , 121, 2005 时的解分别为 198, 0, 1979。

**【分析】**

本题看起来是个数学题, 实则不然。假设所求生成元为  $m$ 。不难发现  $m < n$ 。换句话说, 只需枚举所有的  $m < n$ , 看看有没有哪个数是  $n$  的生成元。

可惜这样做的效率并不高, 因为每次计算一个  $n$  的生成元都需要枚举  $n-1$  个数。有没有更快的方法? 聪明的读者也许已经想到了: 只需一次性枚举 100000 内的所有正整数  $m$ , 标记“ $m$  加上  $m$  的各个数字之和得到的数有一个生成元是  $m$ ”, 最后查表即可。

```
#include <stdio.h>
#include <string.h>
#define maxn 100005
int ans[maxn];

int main() {
    int T, n;
    memset(ans, 0, sizeof(ans));
    for(int m = 1; m < maxn; m++) {
        int x = m, y = m;
        while(x > 0) { y += x % 10; x /= 10; }
        if(ans[y] == 0 || m < ans[y]) ans[y] = m;
    }
    scanf("%d", &T);
    while(T--) {
        scanf("%d", &n);
        printf("%d\n", ans[n]);
    }
    return 0;
}
```

**例题 3-6 环状序列 (Circular Sequence, ACM/ICPC Seoul 2004, UVa1584)**

长度为  $n$  的环状串有  $n$  种表示法, 分别为从某个位置开始顺时针得到。例如, 图 3-4 的环状串有 10 种表示: CGAGTCAGCT, GAGTCAGCTC, AGTCAGCTCG 等。在这些表示法中, 字典序最小的称为“最小表示”。

输入一个长度为  $n$  ( $n \leq 100$ ) 的环状 DNA 串 (只包含 A、C、G、T 这 4 种字符) 的一种表示法, 你的任务是输出该环状串的最小表示。例如, CTCC 的最小表示是 CCCT, CGAGTCAGCT 的最小表示为 AGCTCGAGTC。

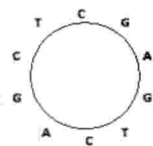


图 3-4 环状串

**【分析】**

本题出现了一个新概念: 字典序。所谓字典序, 就是字符串在字典中的顺序。一般地,



对于两个字符串,从第一个字符开始比较,当某一个位置的字符不同时,该位置字符较小的串,字典序较小(例如,abc 比 bcd 小);如果其中一个字符串已经没有更多字符,但另一个字符串还没结束,则较短的字符串的字典序较小(例如,hi 比 history 小)。字典序的概念可以推广到任意序列,例如,序列 1,2,4,7 比 1,2,5 小。

学会了字典序的概念之后,本题就不难解决了:就像“求  $n$  个元素中的最小值”一样,用变量 ans 表示目前为止,字典序最小串在输入串中的起始位置,然后不断更新 ans。

```
#include<stdio.h>
#include<string.h>
#define maxn 105

//环状串 s 的表示法 p 是否比表示法 q 的字典序小
```

```
int less(const char* s, int p, int q) {
    int n = strlen(s);
    for(int i = 0; i < n; i++)
        if(s[(p+i)%n] != s[(q+i)%n])
            return s[(p+i)%n] < s[(q+i)%n];
    return 0; //相等
}
```

```
int main() {
    int T;
    char s[maxn];
    scanf("%d", &T);
    while(T--) {
        scanf("%s", s);
        int ans = 0;
        int n = strlen(s);
        for(int i = 1; i < n; i++)
            if(less(s, i, ans)) ans = i;
        for(int i = 0; i < n; i++)
            putchar(s[(i+ans)%n]);
        putchar('\n');
    }
    return 0;
}
```

### 3.4 注解与习题

到目前为止,C 语言的核心内容已经全部讲完。理论上,运用前 3 章的知识足以编写大部分算法竞赛程序了。

### 3.4.1 进位制与整数表示

用 ASCII 编码表示字符。下面来探索一下字符在 C 语言中的表示。从正文中可知，有些特殊的字符需要转义才能表达，例如“\n”表示换行，“\”表示反斜杠，“\"表示引号，“\0”表示空字符，那还有哪些转义符呢？如果在网上搜索一下，或者翻阅任何一本 C 语言参考书，就会发现转义字符表中有如下说法。

**提示 3-21：**字符还可以直接用 ASCII 码表示。如果用八进制，应该写成“\o”，“\oo”或“\ooo”（o 为一个八进制数字）；如果用十六进制，应该写成“\xh”（h 为十六进制数字串）。

什么是八进制和十六进制呢？我们平时使用的是“逢十进一”的进位制系统，称为十进制（Decimal System）。而在计算机内部，所有事物都是用“逢二进一”的二进制（Binary System）来表示。从表 3-1 很容易看出二者之间的关系。

表 3-1 十进制和二进制的转换关系

| 十进制 | 0 | 1 | 2  | 3  | 4   | 5   | 6   | 7   |
|-----|---|---|----|----|-----|-----|-----|-----|
| 二进制 | 0 | 1 | 10 | 11 | 100 | 101 | 110 | 111 |

类似地，可以定义八进制和十六进制（注意，在十六进制中，用字符 A~F 表示十进制中的 10~15）。如果操作系统是 Windows，打开“计算器”后，先切换到“科学型”，然后输入一个整数，例如 123，再单击“二进制”按钮，就可以看到其二进制值 1111011、八进制值 173 和十六进制值 7B<sup>①</sup>。而语句“printf(“%d %o %x\n”, a)”将把整数 a 分别按照十进制、八进制和十六进制输出。

**进制转换与移位运算符。**如何把二进制转换为十进制？类似于  $123 = ((1 \times 10) + 2) \times 10 + 3$ ，二进制转换为十进制也可以这样一次添加一位，每次乘以 2： $101_2 = ((1 \times 2 + 0) \times 2 + 1) = 5$ 。在 C 语言中，“乘以 2”也可以写成“<<1”，意思是“左移一位”。类似地，左移 4 位就是乘以  $2^4$ 。

在二进制中，8 位最大整数就是 8 个 1，即  $2^8 - 1$ ，用 C 语言写出来就是  $(1 << 8) - 1$ 。注意括号是必需的，因为“<<”运算符的优先级没有减法高。

**补码表示法。**计算机中的二进制是没有符号的。尽管 123 的二进制值是 1111011，-123 在计算机内并不表示为 -1111011——这个“负号”也需要用二进制位来表示。

“正号和符号”只有两种情况，因此用一个二进制位就可以了。容易想到一个表示“带符号 32 位整数”的方法：用最高位表示符号（0：正数；1：负数），剩下 31 位表示数的绝对值。可惜，这并不是机器内部真正的实现方法。在笔者的机器上，语句“printf(“%u\n”, -1)”的输出是 4294967295<sup>②</sup>。把 -1 换成 -2、-3、-4……后，很容易总结出一个规律：-n 的内部表示是  $2^{32} - n$ 。这就是著名的“补码表示法”（Complement Representation）。

**提示 3-22：**在多数计算机内部，整数采用的是补码表示法。

为什么计算机要用这样一个奇怪的表示方法呢？前面提到的“符号位+绝对值”的方法

<sup>①</sup> 遗憾的是，Linux 下的 GUI 计算器 xcalc 无法进行进制转换。不过很多系统预装了 bc 程序，可以使用“echo 'obase=2; ibase=10; 123 | bc’”把十进制 123 转换成二进制。

<sup>②</sup> 请记住这个整数，它等于  $2^{32} - 1$ 。

哪里不好了？答案是：运算不方便。试想，要计算  $1 + (-1)$  的值（为了简单起见，假设两个数都是带符号 8 位整数）。如果用“符号位+绝对值”法，将要计算  $00000001 + 10000001$ ，而答案应该是  $00000000$ 。似乎想不到什么简单的方法进行这个“加法”。但如果采用补码表示，计算的是  $00000001 + 11111111$ ，只需要直接相加，并丢掉最高位的进位即可。“符号位+绝对值”还有一个好玩的 bug：存在两种不同的 0：一个是  $00000000$ （正 0），一个是  $10000000$ （负 0）。这个问题在补码表示法中不会出现（想一想，为什么）。

学到这里，你能解释“int 类型的最小、最大值”了吗？提示：在通常情况下，int 是 32 位的。

### 3.4.2 思考题

**题目 1（必要的存储量）：**数组可以用来保存很多数据，但在一些情况下，并不需要把数据保存下来。下面哪些题目可以不借助数组，哪些必须借助数组？请编程实现。假设输入只能读一遍。

- ☐ 输入一些数，统计个数。
- ☐ 输入一些数，求最大值、最小值和平均数。
- ☐ 输入一些数，哪两个数最接近。
- ☐ 输入一些数，求第二大的值。
- ☐ 输入一些数，求它们的方差。
- ☐ 输入一些数，统计不超过平均数的个数。

**题目 2（统计字符 1 的个数）：**下面的程序意图在于统计字符串中字符 1 的个数，可惜有瑕疵：

```
#include<stdio.h>
#define maxn 10000000 + 10
int main() {
    char s[maxn];
    scanf("%s", s);
    int tot = 0;
    for(int i = 0; i < strlen(s); i++)
        if(s[i] == 1) tot++;
    printf("%d\n", tot);
}
```

该程序至少有 3 个问题，其中一个导致程序无法运行，另一个导致结果不正确，还有一个导致效率低下。你能找到它们并改正吗？

### 3.4.3 黑盒测试和在线评测系统

**黑盒测试。**算法竞赛一般采取黑盒测试：事先准备好一些测试用例，然后用它们测试选手程序，根据运行结果评分。除了找不到程序（如程序名没有按照比赛规定取，或是放

错位置）、编译错等连程序都没能运行的错误之外，一些典型的错误类型如下：

- ❑ 答案错（Wrong Answer, WA）。
- ❑ 输出格式错（Presentation Error, PE）。
- ❑ 超时（Time Limit Exceeded, TLE）。
- ❑ 运行错（Runtime Error, RE）。

在一些比较严格的比赛中，输出格式错被看成是答案错，而在另外一些比赛中，则会把二者区分开。在运行时，除了程序自身异常退出（例如，除 0、栈溢出、非法访问内存、断言为假、main 函数返回非 0 值）外，还可能是因为超过了评测系统的资源约束（如内存限制、最大输出限制）而被强制中止执行。有的评测系统会把这些情况和一般的运行错误区分开，但在多数情况下会统一归到“运行错”中。

需要注意的是，超时不一定是因为程序效率太低，也可能是其他原因造成的。例如，比赛规定程序应从文件读入数据，但所写程序却正在等待键盘输入。其他原因包括：特殊数据导致程序进入死循环、程序实际上已经崩溃却没异常退出等。

如果上述错误都没有，那么恭喜你，你的程序通过了测试。在 ACM/ICPC 中，这意味着你的程序被裁判接受（accepted, AC），而在分测试点的比赛中，这意味着你拿到了该测试点的分数。

需要注意的是，一些比赛的测试点可以给出“部分分”——如答案正确但不够优，或者题目中有两个任务，选手只成功完成了一个任务等。不管怎样，得分的前提是不超时、没有运行错。只有这样，程序输出才会参与评分。

在线评测系统（Online Judge, OJ）为平时练习和网上竞赛提供了一个很好的平台。事实上，本书中的练习大都通过 OJ 给出。

首先，要向读者介绍的是历史最悠久、最著名的 OJ：西班牙 Valladolid 大学的 UVaOJ，网址为 <http://uva.onlinejudge.org/><sup>①</sup>。除了收录了早期的 ACM/ICPC 区域比赛题目之外，这里还经常邀请世界顶尖的命题者共同组织网上竞赛，吸引了大量来自世界各地的高手同场竞技。

目前，UVaOJ 网站的题库已经包含了一个特殊的分卷（Volume）——AOAPC II，把本书的配套习题按照易于查找和提交的方式集中在一起，并将逐步提供题目的中文翻译和算法提示。根据读者的反馈，网上题库可能在本书的基础上增加一些有价值的题目，并移除一些不太合适的题目，因此建议读者在做题时直接参考 UVaOJ 的 AOAPC 分卷。3.4.2 节的题目中已经给出了 UVa 题目编号。例如，UVa272 就代表 UVa OJ 中编号为 272 的题目。

其他著名的 OJ 包括国内的 ZOJ（浙江大学），POJ（北京大学），HDOJ（电子科技大学）、俄罗斯的 SGU、Timus、波兰的 SPOJ 等。

### 3.4.4 例题一览与习题

本章的 5 道例题全部是竞赛题目，在 UVa 上可以提交，如表 3-2 所示。

<sup>①</sup> 目前的 UVaOJ 网站与 IE 浏览器兼容性不好，推荐使用 Firefox 浏览器。

表 3-2 例题一览

| 类 别    | 题 号      | 题目名称 (英文)         | 备 注        |
|--------|----------|-------------------|------------|
| 例题 3-1 | UVa272   | Tex Quotes        | 输入输出函数详解   |
| 例题 3-2 | UVa10082 | WERTYU            | 常量数组的妙用    |
| 例题 3-3 | UVa401   | Palindromes       | 字符函数; 常量数组 |
| 例题 3-4 | UVa340   | Master-Mind Hints | 用数组统计      |
| 例题 3-5 | UVa1583  | Digit Generator   | 预处理、查表     |
| 例题 3-6 | UVa1584  | Circular Sequence | 字典序        |

从本章开始, 习题全部通过 UVaOJ 给出。由于样例输入输出很占篇幅, 这里通过文字的方式给出例子, 详细的样例输入输出请读者参考原题。在下面的习题中, 前一半的题目几乎只需要“按照题目说的做”, 但后面的题目需要一些思考甚至灵感。

为了保证学习效果, 请至少独立完成 8 道习题。需要特别注意的是, 由于本书前 4 章中的 C 语言程序需要用 C99 编译器, 而 UVa 中的“ANSI C”是指 C89 编译器, 请在提交时选择 C++ 语言。本书前 4 章中介绍的 C 语言全部和 C++ 兼容, 所以源码可以不加修改地用 C++ 编译器编译通过。

#### 习题 3-1 得分 (Score, ACM/ICPC Seoul 2005, UVa1585)

给出一个由 O 和 X 组成的串 (长度为 1~80), 统计得分。每个 O 的得分为目前连续出现的 O 的个数, X 的得分为 0。例如, OOXOXXOOO 的得分为 1+2+0+0+1+0+0+1+2+3。

#### 习题 3-2 分子量 (Molar Mass, ACM/ICPC Seoul 2007, UVa1586)

给出一种物质的分子式 (不带括号), 求分子量。本题中的分子式只包含 4 种原子, 分别为 C, H, O, N, 原子量分别为 12.01, 1.008, 16.00, 14.01 (单位: g/mol)。例如, C<sub>6</sub>H<sub>5</sub>OH 的分子量为 94.108g/mol。

#### 习题 3-3 数数字 (Digit Counting, ACM/ICPC Danang 2007, UVa1225)

把前  $n$  ( $n \leq 10000$ ) 个整数顺次写在一起: 123456789101112... 数一数 0~9 各出现多少次 (输出 10 个整数, 分别是 0, 1, ..., 9 出现的次数)。

#### 习题 3-4 周期串 (Periodic Strings, UVa455)

如果一个字符串可以由某个长度为  $k$  的字符串重复多次得到, 则称该串以  $k$  为周期。例如, abcabcabcabc 以 3 为周期 (注意, 它也以 6 和 12 为周期)。

输入一个长度不超过 80 的字符串, 输出其最小周期。

#### 习题 3-5 谜题 (Puzzle, ACM/ICPC World Finals 1993, UVa227)

有一个 5\*5 的网格, 其中恰好有一个格子是空的, 其他格子各有一个字母。一共有 4 种指令: A, B, L, R, 分别表示把空格上、下、左、右的相邻字母移到空格中。输入初始网格和指令序列 (以数字 0 结束), 输出指令执行完毕后的网格。如果有非法指令, 应输出 “This puzzle has no final configuration.”, 例如, 图 3-5 中执行 ARRBBL0 后, 效果如图 3-6 所示。

|   |   |   |   |   |
|---|---|---|---|---|
| T | R | G | S | J |
| X | D | O | K | I |
| M |   | V | L | N |
| W | P | A | B | E |
| U | Q | H | C | F |

图 3-5 执行 ARRBBL0 前

|   |   |   |   |   |
|---|---|---|---|---|
| T | R | G | S | J |
| X | O | K | L | I |
| M | D | V | B | N |
| W | P |   | A | E |
| U | Q | H | C | F |

图 3-6 执行 ARRBBL0 后

### 习题 3-6 纵横字谜的答案 (Crossword Answers, ACM/ICPC World Finals 1994, UVa232)

输入一个  $r$  行  $c$  列 ( $1 \leq r, c \leq 10$ ) 的网格，黑格用 “\*” 表示，每个白格都填有一个字母。如果一个白格的左边相邻位置或者上边相邻位置没有白格（可能是黑格，也可能出了网格边界），则称这个白格是一个起始格。

首先把所有起始格按照从上到下、从左到右的顺序编号为 1, 2, 3, …，如图 3-7 所示。

|    |    |    |    |    |    |   |
|----|----|----|----|----|----|---|
| 1  | 2  | 3  |    | 4  | 5  | 6 |
|    | 7  |    |    | 8  |    |   |
| 9  |    |    | 10 |    | 11 |   |
| 12 |    |    | 13 | 14 |    |   |
|    | 15 | 16 |    | 17 |    |   |
| 18 |    |    |    | 19 | 20 |   |

图 3-7  $r$  行  $c$  列网格

接下来要找出所有横向单词 (Across)。这些单词必须从一个起始格开始，向右延伸到一个黑格的左边或者整个网格的最右列。最后找出所有竖向单词 (Down)。这些单词必须从一个起始格开始，向下延伸到一个黑格的上边或者整个网格的最下行。输入输出格式和样例请参考原题。

### 习题 3-7 DNA 序列 (DNA Consensus String, ACM/ICPC Seoul 2006, UVa1368)

输入  $m$  个长度均为  $n$  的 DNA 序列，求一个 DNA 序列，到所有序列的总 Hamming 距离尽量小。两个等长字符串的 Hamming 距离等于字符不同的位置个数，例如，ACGT 和 GCGA 的 Hamming 距离为 2（左数第 1, 4 个字符不同）。

输入整数  $m$  和  $n$  ( $4 \leq m \leq 50, 4 \leq n \leq 1000$ )，以及  $m$  个长度为  $n$  的 DNA 序列（只包含字母 A, C, G, T），输出到  $m$  个序列的 Hamming 距离和最小的 DNA 序列和对应的距离。如有多解，要求为字典序最小的解。例如，对于下面 5 个 DNA 序列，最优解为 TAAGATAC。

TATGATAC  
TAAGCTAC  
AAAGATCC  
TGAGATAC  
TAAGATGT

**习题 3-8 循环小数 (Repeating Decimals, ACM/ICPC World Finals 1990, UVa202)**

输入整数  $a$  和  $b$  ( $0 \leq a \leq 3000$ ,  $1 \leq b \leq 3000$ ), 输出  $a/b$  的循环小数表示以及循环节长度。例如  $a=5$ ,  $b=43$ , 小数表示为  $0.(116279069767441860465)$ , 循环节长度为 21。

**习题 3-9 子序列 (All in All, UVa 10340)**

输入两个字符串  $s$  和  $t$ , 判断是否可以从  $t$  中删除 0 个或多个字符 (其他字符顺序不变), 得到字符串  $s$ 。例如,  $abcde$  可以得到  $bce$ , 但无法得到  $dc$ 。

**习题 3-10 盒子 (Box, ACM/ICPC NEERC 2004, UVa1587)**

给定 6 个矩形的长和宽  $w_i$  和  $h_i$  ( $1 \leq w_i, h_i \leq 1000$ ), 判断它们能否构成长方体的 6 个面。

**习题 3-11 换低挡装置 (Kickdown, ACM/ICPC NEERC 2006, UVa1588)**

给出两个长度分别为  $n_1, n_2$  ( $n_1, n_2 \leq 100$ ) 且每列高度只为 1 或 2 的长条。需要将它们放入一个高度为 3 的容器 (如图 3-8 所示), 问能够容纳它们的最短容器长度。



图 3-8 高度为 3 的容器

**习题 3-12 浮点数 (Floating-Point Numbers, UVa11809)**

计算机常用阶码-尾数的方法保存浮点数。如图 3-9 所示, 如果阶码有 6 位, 尾数有 8 位, 可以表达的最大浮点数为  $0.11111111_2 \times 2^{111111_2}$ 。注意小数点后第一位必须为 1, 所以一共有 9 位小数。

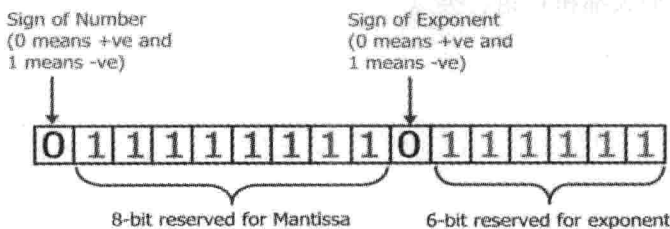


图 3-9 阶码-尾数保存浮点数

这个数换算成十进制之后就是  $0.998046875 \times 2^{63} = 9.205357638345294 \times 10^{18}$ 。你的任务是根据这个最大浮点数, 求出阶码的位数  $E$  和尾数的位数  $M$ 。输入格式为  $AeB$ , 表示最大浮点数为  $A \times 10^B$ 。  $0 < A < 10$ , 并且恰好包含 15 位有效数字。输入结束标志为  $0e0$ 。对于每组数据, 输出  $M$  和  $E$ 。输入保证有唯一解, 且  $0 \leq M \leq 9$ ,  $1 \leq E \leq 30$ 。在本题中,  $M+E+2$  不必为 8 的整数倍。

**3.4.5 小结**

本节介绍的语法和库函数都是很直观的, 但是书中的程序理解起来比第 2 章复杂了很多, 原因在于变量突然多了很多。每当用到  $a[i]$  或者  $s[i]$  这样的元素时, 应该问自己: “ $i$  等于多少? 它有什么实际含义吗?” 作为数组下标,  $i$  经常代表 “当前考虑的位置”, 或者与



另一个下标  $j$  一起表示“当前考虑的子串的起点和终点”。

数组和字符串往往意味着大数据量，而处理大数据量时经常会遇到“访问非法内存”的错误。在语法上，C 语言并不禁止程序访问非法内存，但后果难料。这在理论上可以通过在访问数组前检查下标是否合法来缓解，但程序会比较累赘；另一种技巧是适当把数组空间定义得较大，特别是不清楚数组应该开多大时。只要内存够用，开大一点没关系。顺便说一句，数组的大小可以用 `sizeof` 在编译时获得（它不是一个函数），它经常被用在 `memset`、`memcpy` 等函数中。有的函数并没有做大小检查，因而存在缓冲区溢出漏洞。本章中只讲了 `gets`，但其实 `strcpy` 也有类似问题——如果源字符串并不是以“\0”结尾的，复制工作将可能覆盖到缓冲区之外的内存。这也提醒我们：如果按照自己的方式处理字符串，千万要保证它以“\0”结尾。

在数组和字符串处理程序中，下标的计算是极为重要的。为了方便，很多人喜欢用“++”等可以修改变量（有副作用）的运算符，但千万注意保持程序的可读性。一个保守的做法是如果使用这种运算符，被影响的变量在整个表达式中最多出现一次（例如，`i = i++` 就是不允许的）。

理解字符编码对于正确地使用字符串是至关重要的。算法竞赛中涉及的字符一般是 ASCII 表中的可打印字符。对于中文的 GBK 编码，简单的实验将得出这样的结论：如果 `char` 值为正，则是西文字符；如果为负，则是汉字的前一半（这时需要再读一个 `char`）。这个结论并不是普遍成立的（在某些环境下，`char` 类型是非负的），但在大多数情况下，这样做是可行的。关于字符，另一个有意思的知识是转义序列——几乎所有编程语言都定义了自己的转义序列，但大都和 C 语言类似。